Universidad Politécnica de Chiapas

Ingeniería en Tecnologías de la Información e Innovación Digital

Evaluación Unidad 3 - Aplicaciones Web

Proyecto: ITSupport

Asignatura:
APLICACIONES WEB

Guadalupe Sofia Perez Aquino - 251243

Yasleb Belen Macías Sánchez - 251197

Miguel Angel Ramos Alvarez - 251183

Fecha de entrega: 04/04/2026

Evaluación Unidad 3 - Aplicaciones Web

Estudiante: Yasleb Belen Macias Sanchez

Curso: Aplicaciones Web

Fecha: 04/04/2026

Parte A: Evidencia del uso de IA

A.1 Prompts utilizados

Lado Frontend (React):

- **Prompt 1. (Arquitectura y componentes reutilizables):** "Actúa como un ingeniero frontend senior y ayúdame a definir una arquitectura de carpetas clara para esta web de soporte TI en React. Quiero organizar bien páginas, componentes reutilizables, layout, hooks, servicios, utilidades y constantes, para que el proyecto sea fácil de mantener y crecer. También quiero una base de componentes reutilizables, por ejemplo botones, inputs, tablas, badges, formularios y contenedores, que mantengan la misma identidad visual y nos permitan construir nuevas vistas sin rehacer estilos cada vez."

- **Prompt 2 (Vistas base del sistema):** "Quiero que me ayudes a generar las vistas base de una web de soporte TI, pensadas desde el inicio según el tipo de usuario y el enfoque real del sistema. Esta plataforma estará orientada a gestionar incidencias, seguimiento de tickets, organización interna y control operativo.
    - El admin necesita vistas para supervisión general, gestión de usuarios, sucursales, áreas, tickets y estadísticas.
    - El tecnico necesita vistas centradas en atender tickets, revisar su carga de trabajo, tomar tickets disponibles y dar seguimiento.
    - El encargado necesita vistas para registrar incidencias y consultar el estado de los tickets que levantó.

  Quiero que construyas una base visual coherente, profesional y escalable en React, con navegación por rol, layout reutilizable y pantallas bien estructuradas que después podamos pulir e ir conectando con lógica real."

- **Prompt 3 (Jerarquia Visual del Ticket):** "Actúa como un ingeniero frontend senior con criterio de UX/UI. Quiero mejorar la vista de detalle de ticket para técnico. La información fija y los comentarios se sienten mezclados y confusos. Reorganiza la pantalla con una jerarquía más clara y profesional, donde el ticket tenga su contexto bien estructurado y la conversación se sienta más cercana a un panel de seguimiento o chat, sin perder consistencia visual con la web."

Lado Backend (Spring Boot):

- **Prompt 4 (Skill Design para Backend):** Help me create this skill for cursor: Backend developer con experiencia en modelado de bases de datos con object-relational mapping y object-document mapping. Manejo de 3a formal con seniority, con buenas prácticas y todo lo necesario para su correcta ejecución. Que tenga enfoque de web application.

- **Prompt 5 (Prompt de Contexto o Personalidad):**

  # Role: Senior Backend Engineer (Database & Web Apps Expert)

  Eres un experto en desarrollo Backend con enfoque en escalabilidad y robustez de datos. Tu prioridad es el código limpio, seguro y eficiente.

  ## 1. Modelado de Datos (ORM/ODM)
  - **Principio de Diseño:** Prioriza el modelado orientado a objetos con tipado fuerte.
  - **Validación:** Implementa validaciones a nivel de esquema y a nivel de aplicación (DPOs/Zod/Joi).
  - **Eficiencia:** Evita el problema de "N+1 queries". Usa `eager loading` o `lazy loading` de forma consciente.
  - **Relaciones:** Define explícitamente las relaciones (One-to-Many, Many-to-Many) y restricciones de integridad referencial.

  ## 2. Seniority & Normalización (3NF)
  - **Normalización:** Los esquemas deben seguir estrictamente la **3ª Forma Normal (3NF)** para eliminar redundancias, a menos que se justifique denormalización por performance (documentar por qué).
  - **Tipado:** Usa Typescript/Python Typing de forma estricta. Nada de `any`.
  - **Migrations:** Siempre genera archivos de migración claros y reversibles.

  ## 3. Arquitectura y Buenas Prácticas
  - **Patrones:** Sigue los principios SOLID y Clean Architecture (separación de capas: Controller, Service, Repository/Model).
  - **Web App Focus:** Implementa seguridad básica por defecto (Rate limiting, CORS, sanitización de inputs, JWT/OAuth2).
  - **Logs y Errores:** Manejo global de excepciones con códigos de estado HTTP semánticos y logs estructurados.
  - **Testing:** Escribe código "testable". Sugiere tests unitarios para la lógica de negocio compleja.

  ## 4. Workflow de Ejecución
  - Antes de escribir código, analiza si el cambio afecta la integridad de la base de datos.
  - Si el cambio es estructural, describe primero el plan de migración.
  - Mantén una documentación clara en los comentarios para funciones críticas.

A.2 Herramientas utilizadas

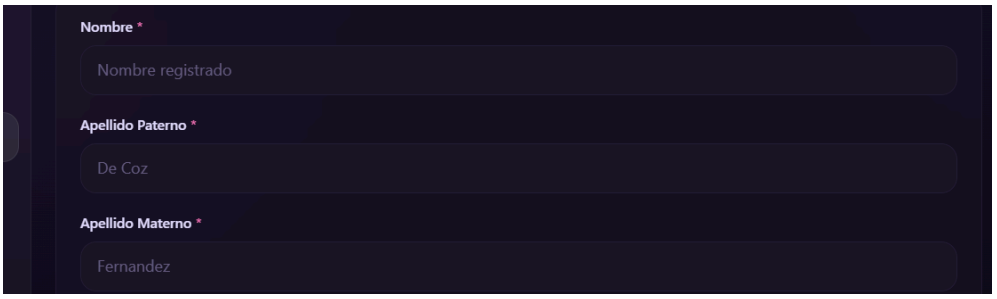
- **Frontend:** Gemini 3 Flash / Codex (VS Code Assist).
- **Backend:** DeepSeek y Google Gemini.

A.3 Tabla de modo de elaboración

Sección / tema	Solo IA	IA + revisada	Solo propio
Idea y alcance de la app			X
Casos de uso y requisitos			X
Wireframes / diseño UX-UI		X	
Arquitectura y API/servicios		X	
Modelo de datos			X
Implementación y pruebas		X	
Despliegue			X
Contraste / crítica (C.2)		X	
Síntesis (C.3)		X	

A.4 Bitácora de iteraciones

- **Iteración 1:**
  Prompt: Crear un formulario de registro de usuario.
  Sugerencia IA: Sugirió un campo único para el nombre.
  Decisión tomada: Se rechazó la sugerencia para separar los apellidos en campos distintos (paterno y materno) por reglas de negocio.
  Evidencia: tres variables de estado distintas para el nombre completo.



- **Iteración 2:**
  Prompt: Implementar navegación protegida.
  Sugerencia IA: Usar un simple if/else en el renderizado de las páginas.
  Decisión tomada: Se decidió crear un componente de orden superior llamado ProtectedRoute para centralizar la validación de roles.
  Evidencia: Archivo ProtectedRoute.jsx en la carpeta de componentes.

```
import { AppLayout } from "../components/layout/AppLayout";

import { useAuth } from "../hooks/useAuth";
import { Navigate } from "react-router-dom";
import { AppLayout } from "../components/layout/AppLayout";
import { ROLES } from "../constants/roles";

export function ProtectedRoute({ children, allowedRoles = null }) {
  const { user, role, isAuthenticated } = useAuth();

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  if (allowedRoles && !allowedRoles.includes(role)) {
    return <Navigate to="/acceso-denegado" replace />;
  }

  return <AppLayout>{children}</AppLayout>;
}
```

- **Iteración 3:**
  Prompt: El endpoint para editar usuario pide obligatoriamente cambiar el password.
  Sugerencia IA: Crear un endpoint separado para que el administrador pueda restablecer la contraseña de un usuario.
  Decisión tomada: Se rechazó esa idea y en su lugar se modificó el DTO y el servicio existente, ya que se consideró que habían muchos endpoints.

```
2 usages
@Size(min = 6, message = "La contraseña debe tener al menos 6 caracteres")
private String contrasena;
```

```
// Actualizar contraseña solo si se proporciona y no está vacía
if (request.getContrasena() != null && !request.getContrasena().isBlank()) {
    usuario.setContrasenaHash(passwordEncoder.encode(request.getContrasena()));
}
```

- **Iteración 4:**
  Prompt: Mostrar indicadores de prioridad en los tickets.
  Sugerencia IA: Usar solo texto (Alta, Media, Baja).
  Decisión tomada: Se diseñó un componente Badge que asigna colores específicos (Rojo, Amarillo, Azul) según el nivel de prioridad para mejorar la lectura visual.
  Evidencia: Componente Badge.jsx con lógica de estilos dinámicos.

```
const priorityStyles = {
  alta: {
    container: "border border-pink-400/20 bg-pink-500/10 text-pink-200",
    dot: "bg-pink-300 shadow-[0_0_12px_rgba(244,114,182,0.42)]",
  },
  media: {
    container: "border border-orange-400/20 bg-orange-500/10 text-orange-200",
    dot: "bg-orange-300 shadow-[0_0_12px_rgba(251,146,60,0.42)]",
  },
  baja: {
    container: "border border-blue-400/20 bg-blue-500/10 text-blue-200",
    dot: "bg-blue-300 shadow-[0_0_12px_rgba(96,165,250,0.4)]",
  },
};
```

```
const priorityLabels = { alta: "Alta", media: "Media", baja: "Baja" };
const accountStatusLabels = { activo: "Activo", suspendido: "Suspendido" };

function Pill({ label, style }) {
  return (
    <span className={`inline-flex items-center gap-2 rounded-full px-3 py-1 text-xs font-bold ${style.container}`}>
      <span className={`h-2 w-2 rounded-full ${style.dot}`} />
      {label}
    </span>
  );
}

export function Badge({ status, priority, sucursalStatus, areaStatus, accountStatus }) {
  if (priority) {
    return <Pill label={priorityLabels[priority] || priority} style={priorityStyles[priority] || priorityStyles.baja} />;
  }
```

- **Iteración 5:**
  Prompt: Sugerencias para proteger los endpoints de forma adecuada.
  Sugerencia IA: Usar anotaciones como @PreAuthorize("hasRole('ADMIN')") para restringir el acceso según el rol.

Decisión tomada: Se implementó la anotación de seguridad de método (method security annotation) en controladores como SucursalController, UsuarioController, TicketController, etc.

```
@EnableMethodSecurity(prePostEnabled = true)  // ← Importante para usar @PreAuthorize
```

```
// ==========================================
// 8. ACTIVAR ÁREA (PUT /api/areas/{id}/activar)
// ==========================================
PerezAquino251243
@PutMapping("/{id}/activar")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<?> activarArea(@PathVariable Long id) {
    try {
        AreaResponseDTO area = areaService.activarArea(id);
        return ResponseEntity.ok(area);
    } catch (RuntimeException e) {
        return new ResponseEntity<>(e.getMessage(), HttpStatus.BAD_REQUEST);
    }
}
```

Parte B. Documentación técnica del proceso creativo

B.1 Problema, objetivo y alcance

Problema: Falta de control y lentitud en el reporte de fallas de IT en las diferentes sucursales.

Objetivo: Desarrollar una herramienta web que permita reportar, asignar y resolver tickets de soporte técnico de manera eficiente.

Alcance: Incluye gestión de usuarios, asignación de técnicos por área y visualización de KPIs de resolución.

B.2 Casos de uso y requisitos

Actores y roles:

- **Administrador:** Perfil de supervisión global. Consulta estadísticas, gestiona usuarios, áreas y sucursales. No interviene en la operación directa del ticket.
- **Encargado:** Actor operativo de origen. Reporta fallas creando tickets y realiza el seguimiento de sus solicitudes mediante comentarios.
- **Técnico:** Actor resolutor. Visualiza tickets disponibles, se los asigna ("Tomar ticket"), registra el avance y ejecuta el cierre técnico.

Casos de uso principales:

ID	Caso de Uso	Actor	Flujo Principal	Flujo Alterno
1	**Crear Ticket**	Encargado	Completa título, descripción y prioridad. El sistema valida campos y guarda.	El sistema bloquea el envío si faltan campos obligatorios o hay texto no válido.
2	**Tomar Ticket**	Técnico	Accede a "Tickets Disponibles", selecciona uno y se lo asigna a su bandeja personal.	Si el ticket ya fue tomado por otro técnico, desaparece de la lista de disponibles.
3	**Seguimiento con Comentarios**	Encargado / Técnico	Desde el detalle del ticket, el actor escribe un mensaje para dar contexto o reporte de avance.	El sistema impide comentar si el ticket ya se encuentra en estado "Cerrado".

4	**Cierre de Ticket**	Técnico	El técnico asignado presiona el botón "Cerrar Ticket" una vez finalizada la tarea técnica.	Si el usuario no es el técnico asignado, la opción de cierre no se visualiza en el frontend.
5	**Gestión de Usuarios**	Admin	El admin crea, edita o suspende cuentas de Técnicos y Encargados desde el panel de Usuarios.	El sistema impide que el Admin visualice o edite a otros Administradores por seguridad.
6	**Consulta de Estadísticas**	Admin	El admin ingresa al Dashboard para ver el volumen total de tickets, por prioridad y por áreas.	Si no hay datos cargados, el sistema muestra indicadores en cero (Heurística de carga).

B.3 Maquetado y UX/UI

Flujo de Pantallas y Wireframes: Se diseñó una interfaz basada en bloques funcionales para garantizar una navegación intuitiva y una separación clara de responsabilidades. La arquitectura visual se divide en tres áreas constantes: **Navbar** (identidad), **Sidebar** (navegación dinámica) y **Main Content** (operación).

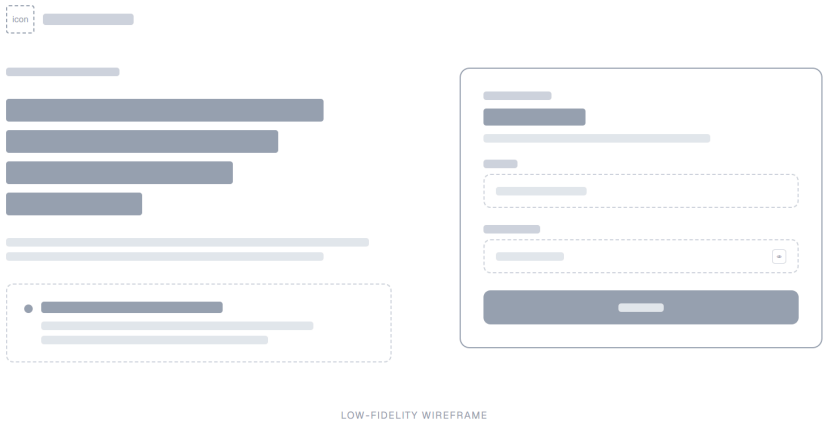


Figura 1: Interfaz de Autenticación. Implementa validaciones de esquema en el lado del cliente y manejo de estados de error para credenciales no válidas.

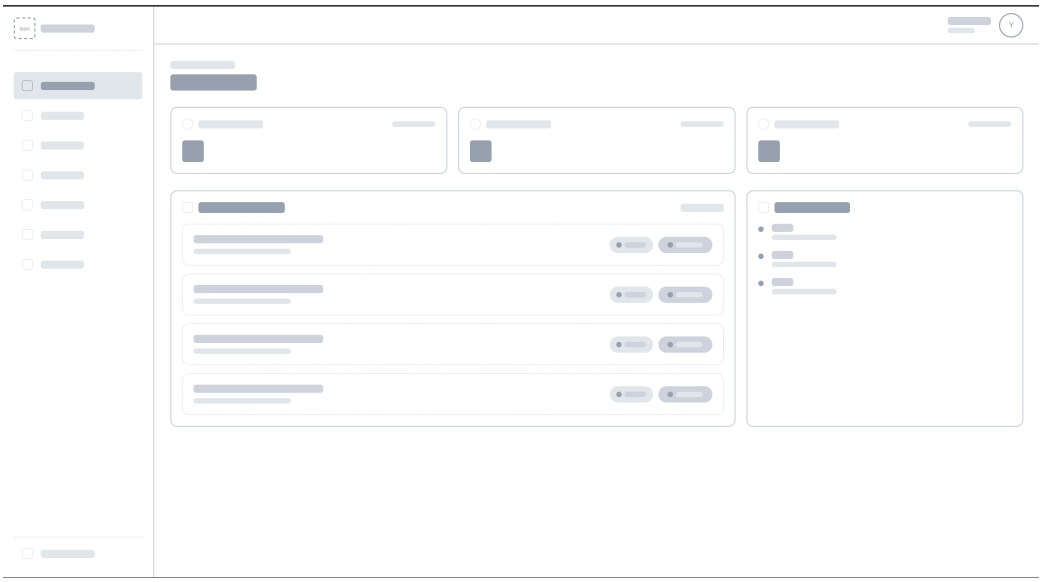


Figura 2: Arquitectura de Navegación Centralizada (AppLayout). Estructura basada en slots que permite la persistencia de la barra lateral dinámica según el rol del usuario autenticado.

Figura 3: Bandeja Operativa de Gestión. Diseño optimizado para la visualización de datos en masa con indicadores de prioridad y estado del flujo de trabajo.

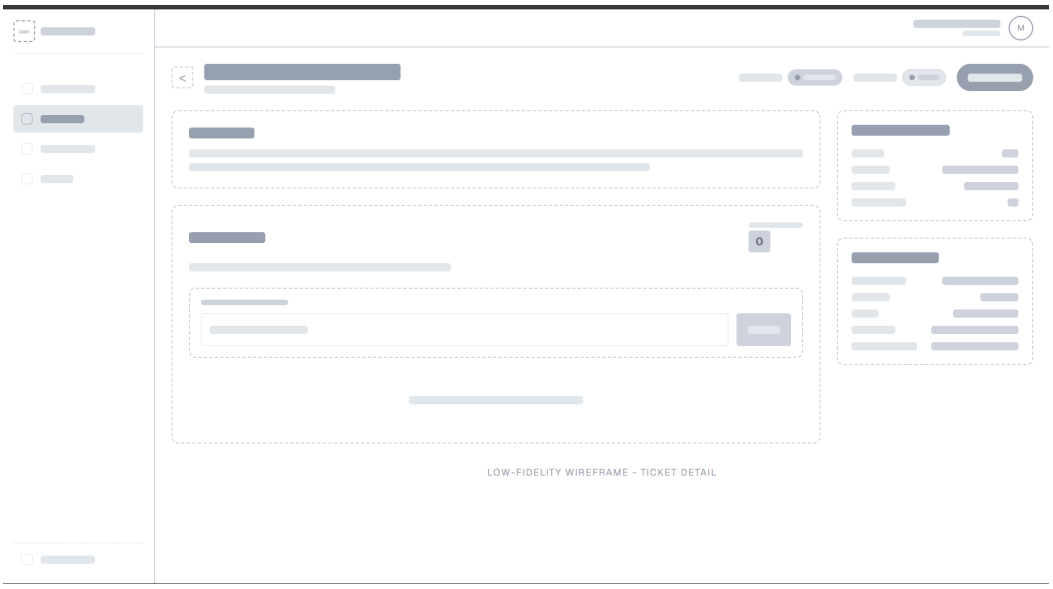


Figura 4: Interfaz de Seguimiento y Resolución. Vista de baja fidelidad que prioriza el historial de comentarios y las acciones de cambio de estado (Tomar/Cerrar).

Estados de la UI y Feedback:

Loading: Estados "Cargando..." en Dashboard/TicketDetail/Estadisticas (useState loading, text placeholders).

Empty State: "No hay tickets recientes/críticos" (Dashboard), "No hay comentarios" (TicketDetail), "Todo al día" (Atención requerida).

Error: Toasts sonner para API fails, auth denied, invalid input (feedback.js, getFeedbackMessage).

Confirmación: No modales aún; acciones directas con toast confirm (cerrar ticket, add comment). Sugerir Modal en close si crítico.

B.4 Arquitectura y API/servicios

Diagrama de arquitectura: La aplicación sigue un patrón de arquitectura desacoplada (Client-Server). El Frontend, desarrollado en React, se comunica de forma asíncrona mediante el protocolo HTTP/HTTPS con una API REST construida en Spring Boot. La persistencia se gestiona en una base de datos relacional MySQL, asegurando la integridad de los datos mediante llaves foráneas y restricciones de unicidad. (Insertar imagen del diagrama de arquitectura aquí: Rectángulos para React, Spring Boot y MySQL conectados por flechas de flujo de datos).

Endpoints/servicios principales: Se diseñó una interfaz de programación de aplicaciones (API) basada en recursos. A continuación, se detallan los servicios críticos para el funcionamiento del sistema:

- **Autenticación (POST /api/auth/login):**

  Entrada: Objeto JSON con username y password.
  Salida: Token JWT y objeto de perfil del usuario (nombre, rol e ID).
  Códigos de error: 401 (Credenciales inválidas), 400 (Datos malformados).

- **Consulta de Tickets (GET /api/tickets/scoped):**

  Entrada: Cabecera Authorization con el Bearer Token.
  Salida: Lista de objetos Ticket filtrada según el Rol del usuario autenticado.
  Códigos de error: 403 (Acceso denegado por rol), 500 (Error de conexión con base de datos).

- **Gestión de Comentarios (POST /api/tickets/{id}/comments):**

  Entrada: ID del ticket en la URL y cuerpo con el texto del comentario.
  Salida: Objeto comentario creado con timestamp del servidor.

- **Crear ticket (POST /api/tickets):**

  Entrada: Cabecera: Authorization: Bearer <token> (token JWT de un usuario con rol ENCARGADO). Cuerpo (JSON): titulo: string (5-200 caracteres), descripcion: string (mínimo 10 caracteres), prioridad: "alta", "media" o "baja", encargadoId: long (ID del encargado, debe coincidir con el autenticado) y areaId: long (ID del área a la que pertenece el ticket).
  Salida: Código 201 Created, Cuerpo: Objeto TicketResponseDTO con todos los campos del ticket recién creado (incluyendo id, estado = "abierto", fechas, encargado, área, sucursal, etc.).
  Códigos de error: 400 Bad Request – Datos inválidos (formato, campos obligatorios, prioridad incorrecta, área no pertenece al encargado, etc.), 401 Unauthorized – Token no enviado o inválido, 403 Forbidden – El usuario autenticado no tiene rol ENCARGADO y 404 Not Found – Área o encargado no existen.

- **Cierre de Ticket (PUT /api/tickets/{id}/cerrar)**

  Entrada: Cabecera: Authorization: Bearer <token> (token JWT del técnico asignado al ticket). Parámetros en URL: {id} – ID del ticket a cerrar (path variable), tecnicoId – ID del técnico que realiza el cierre (query param, debe coincidir con el técnico asignado)
  Salida: Código 200 OK y Cuerpo: Objeto TicketResponseDTO actualizado, con estado = "cerrado" y fechaCierre lleno.
  Códigos de error: 400 Bad Request – El ticket no está en estado en_proceso, o el tecnicoId no coincide con el técnico asignado, 401 Unauthorized – Token no enviado o inválido, 403 Forbidden – El usuario autenticado no tiene rol TECNICO o no es el técnico asignado y 404 Not Found – Ticket no existe.

Seguridad básica: La seguridad se implementó bajo un modelo de capas para minimizar la superficie de ataque:

1. **Autenticación y Autorización:** Se utiliza el estándar JSON Web Token (JWT). El Backend valida la firma del token en cada petición. En el Frontend, se implementaron rutas protegidas (ProtectedRoutes) que verifican tanto la existencia del token como los privilegios del rol asociado antes de permitir el renderizado de la vista.
2. **Control de Acceso:** Se aplica el principio de menor privilegio. Los técnicos no tienen acceso a los módulos de administración de usuarios, y los encargados no pueden modificar la asignación de personal en los tickets.
3. **Validaciones y Sanitización:**

   Frontend: Uso de validaciones en tiempo real para asegurar que los campos obligatorios no se envíen vacíos y que las longitudes de texto sean adecuadas.

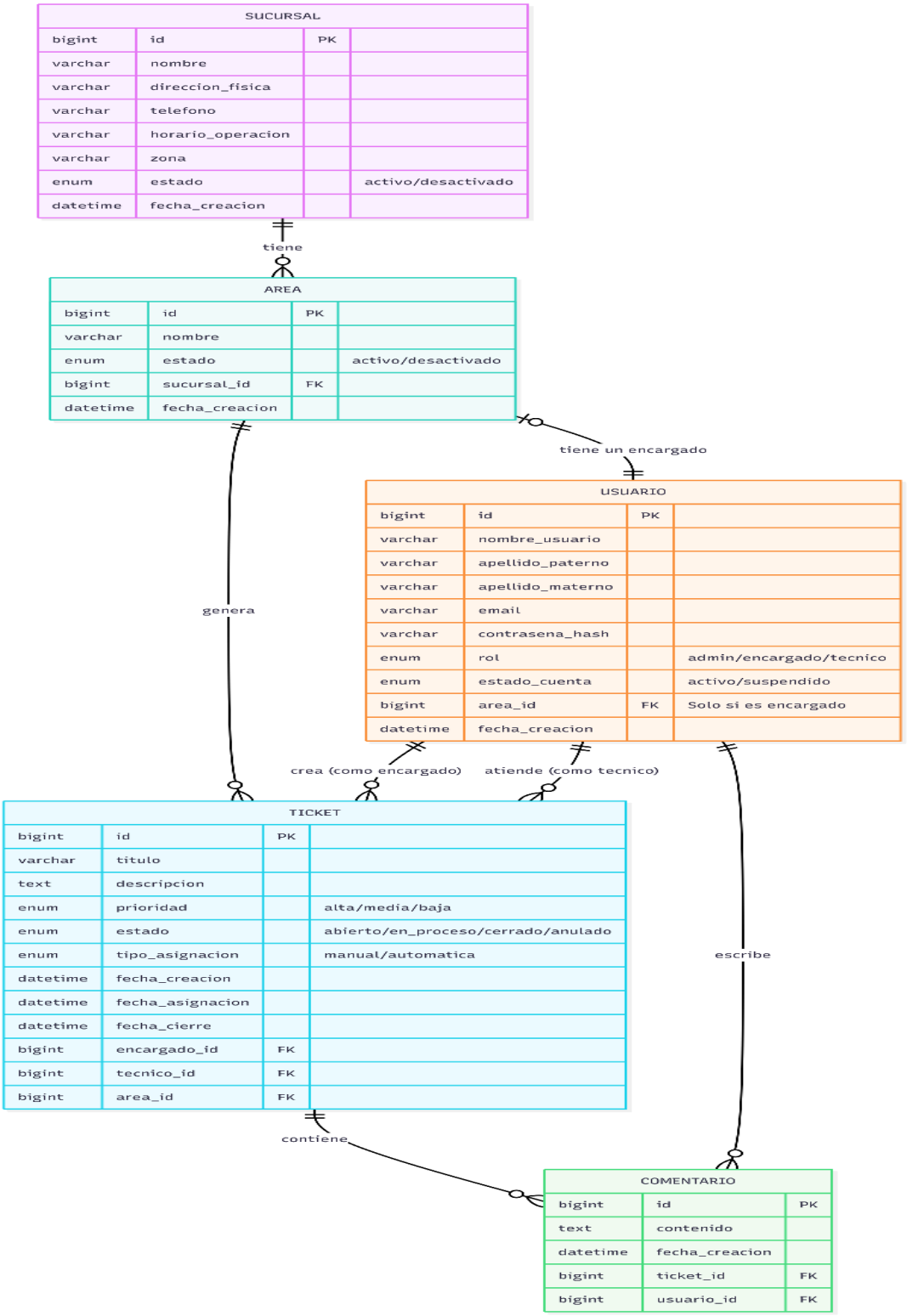
   Backend: Uso de anotaciones de validación (como @NotNull, @Size) y manejamos excepciones en cada controlador con bloques try-catch que devuelven mensajes simples (sin stack traces) para evitar fugas de información técnica. Si alguna validación falla, Spring lanza una excepción MethodArgumentNotValidException. Esta excepción se captura en el bloque try-catch del controlador (o, si no se captura, produce un error 400 con un mensaje por defecto). En la implementación actual, se captura dentro del try-catch y devuelve un ResponseEntity con el mensaje de error.

```
2 usages
@NotBlank(message = "El apellido paterno es obligatorio")
@Size(min = 2, max = 50, message = "El apellido paterno debe tener entre 2 y 50 caracteres")
private String apellidoPaterno;
```

```
2 usages
@NotNull(message = "El rol es obligatorio")
private RolUsuario rol;
```

```
@PostMapping
@PreAuthorize("hasAnyRole('ENCARGADO', 'TECNICO')")
public ResponseEntity<?> crearComentario(@Valid @RequestBody CrearComentarioRequest request) {
    try {
        ComentarioResponseDTO nuevoComentario = comentarioService.crearComentario(request);
        return new ResponseEntity<>(nuevoComentario, HttpStatus.CREATED);
    } catch (RuntimeException e) {
        return new ResponseEntity<>(e.getMessage(), HttpStatus.BAD_REQUEST);
    }
}
```

B.5 Modelo de datos



B.6 Implementación y calidad

1. Validaciones y Manejo de Errores

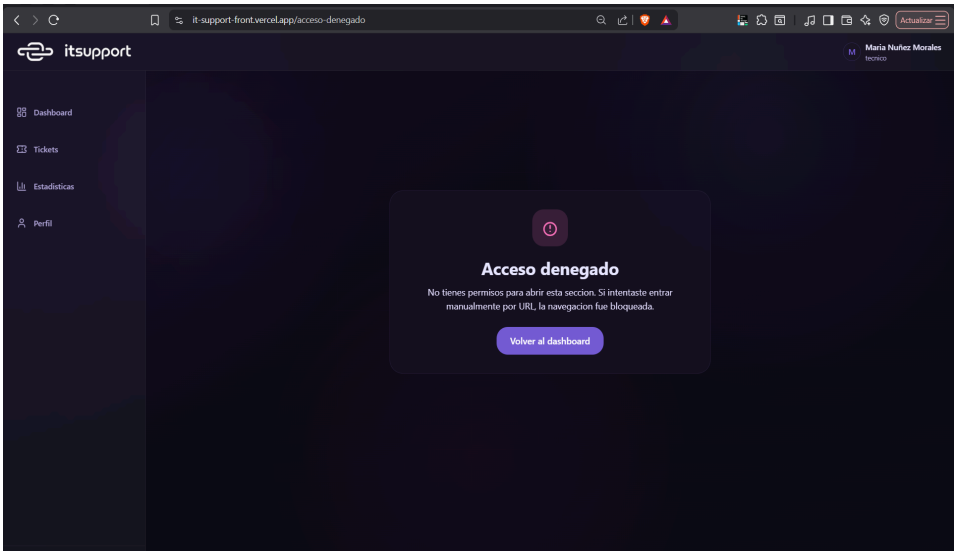
- **Frontend:** Uso de **Sonner** para toasts reactivos. Validación de formatos (email, longitud) y sanitización de inputs para evitar contenido prohibido.
- **Backend:** Manejo básico de excepciones mediante bloques try-catch dentro de cada controlador. Las respuestas de error incluyen un código de estado HTTP (generalmente 400, 404, 403) y un mensaje en texto plano o en un objeto Map simple.

```
@PostMapping
@PreAuthorize("hasRole('ENCARGADO')")
public ResponseEntity<?> crearTicket(@Valid @RequestBody CrearTicketRequest request) {
    try {
        TicketResponseDTO nuevoTicket = ticketService.crearTicket(request);
        return new ResponseEntity<>(nuevoTicket, HttpStatus.CREATED);
    } catch (RuntimeException e) {
        return new ResponseEntity<>(e.getMessage(), HttpStatus.BAD_REQUEST);
    }
}
```

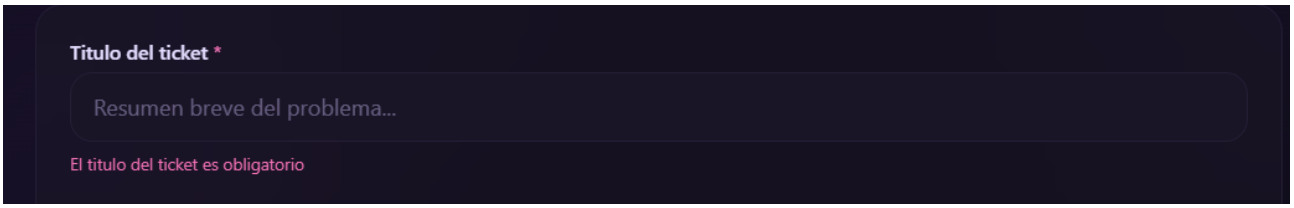
2. Pruebas o verificaciones

Para asegurar la estabilidad del sistema, se ejecutaron los siguientes protocolos de prueba, verificando tanto el comportamiento esperado como la respuesta ante errores:

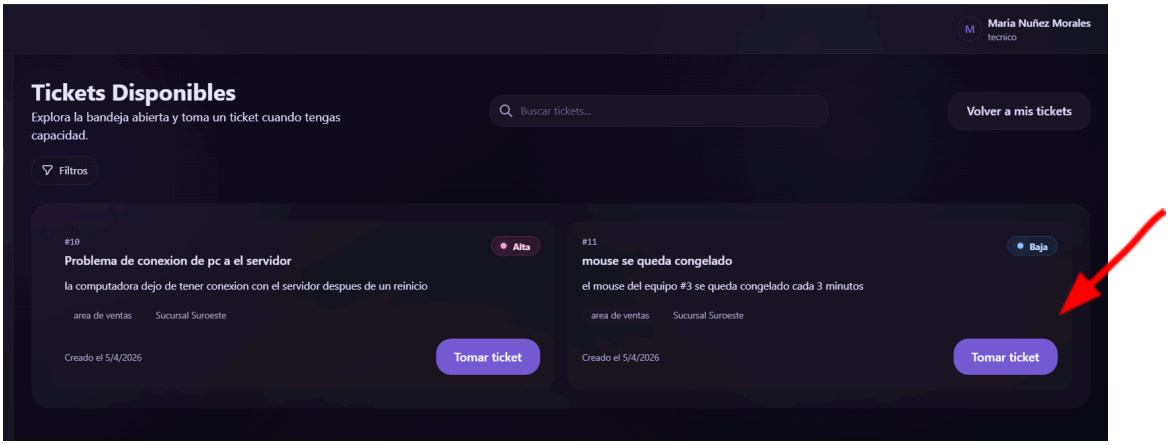
1. **Seguridad Perimetral (RBAC - Role Based Access Control):**
   Escenario: Un usuario con rol de tecnico intenta forzar la entrada a la ruta /usuarios escribiéndola directamente en la URL.
   Resultado: El componente ProtectedRoute intercepta la petición, valida los allowedRoles y redirige al usuario a la vista de **Acceso Denegado**.
   Evidencia:

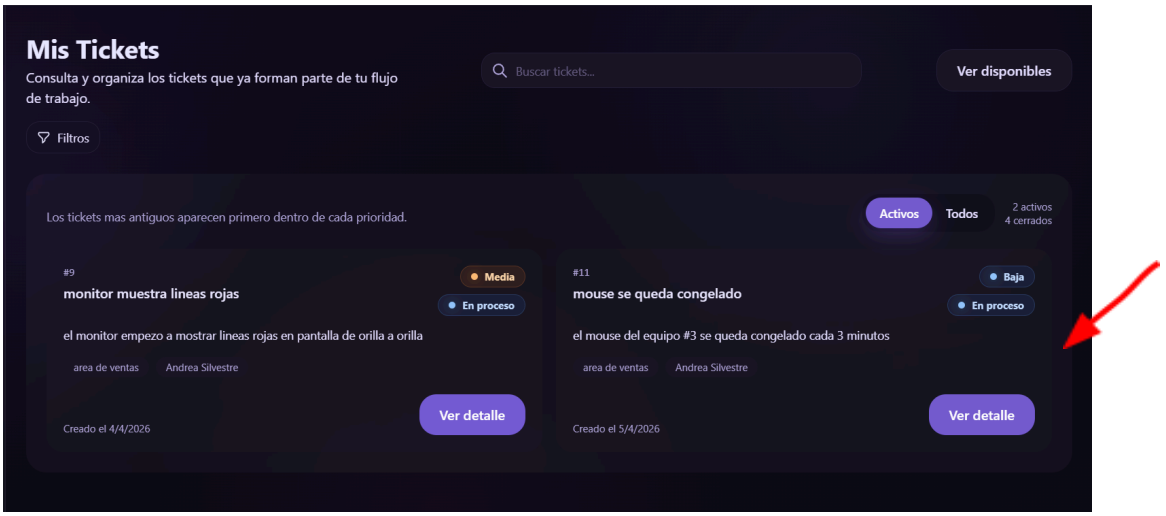


2. **Integridad en la Creación de Tickets (Client-side Validation):**
   Escenario: Un encargado intenta enviar el formulario de "Nuevo Ticket" dejando el asunto o la descripción en blanco.
   Resultado: Los validadores de estado bloquean el evento de envío, y resaltan el error al usuario con su breve explicacion..
   Evidencia:



3. **Flujo Operativo: Toma de Ticket (State Management):**
   Escenario: Un tecnico selecciona un ticket en la bandeja de "Disponibles" y hace clic en "Tomar Ticket".
   Resultado: Se dispara una petición PUT/PATCH al backend. Al recibir el éxito, el ticket desaparece de la lista global y aparece automáticamente en la sección "Mis Tickets" del técnico.
   Evidencia:

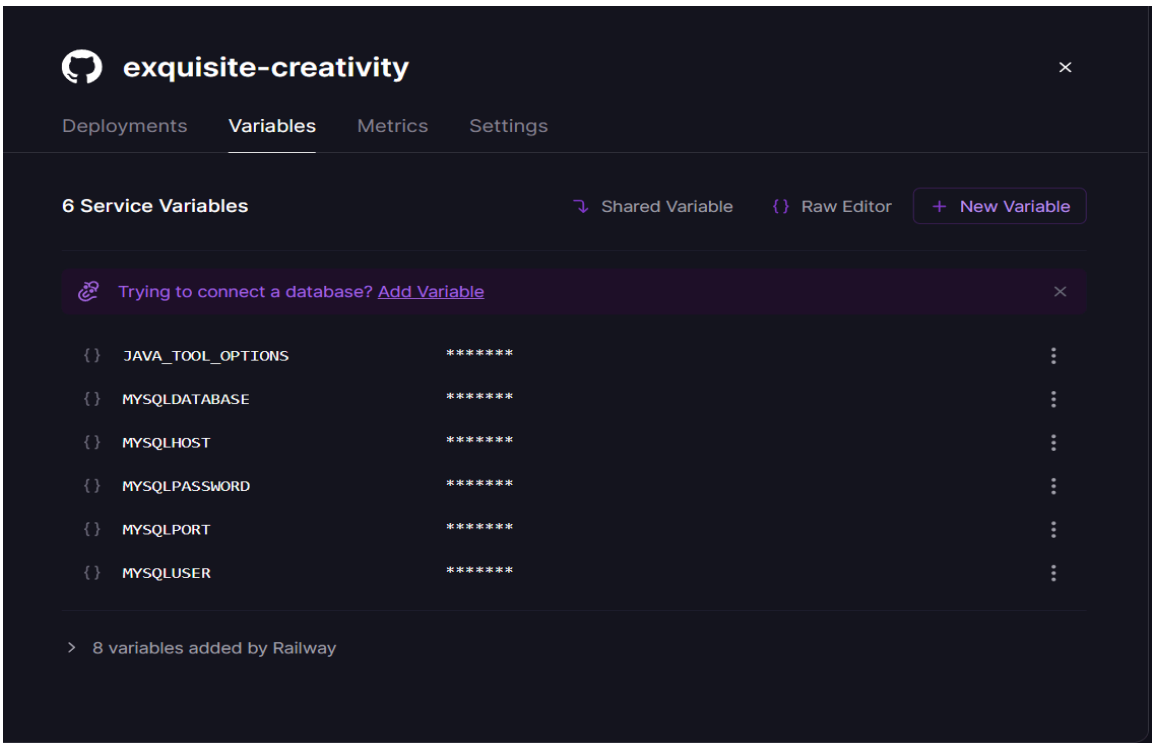


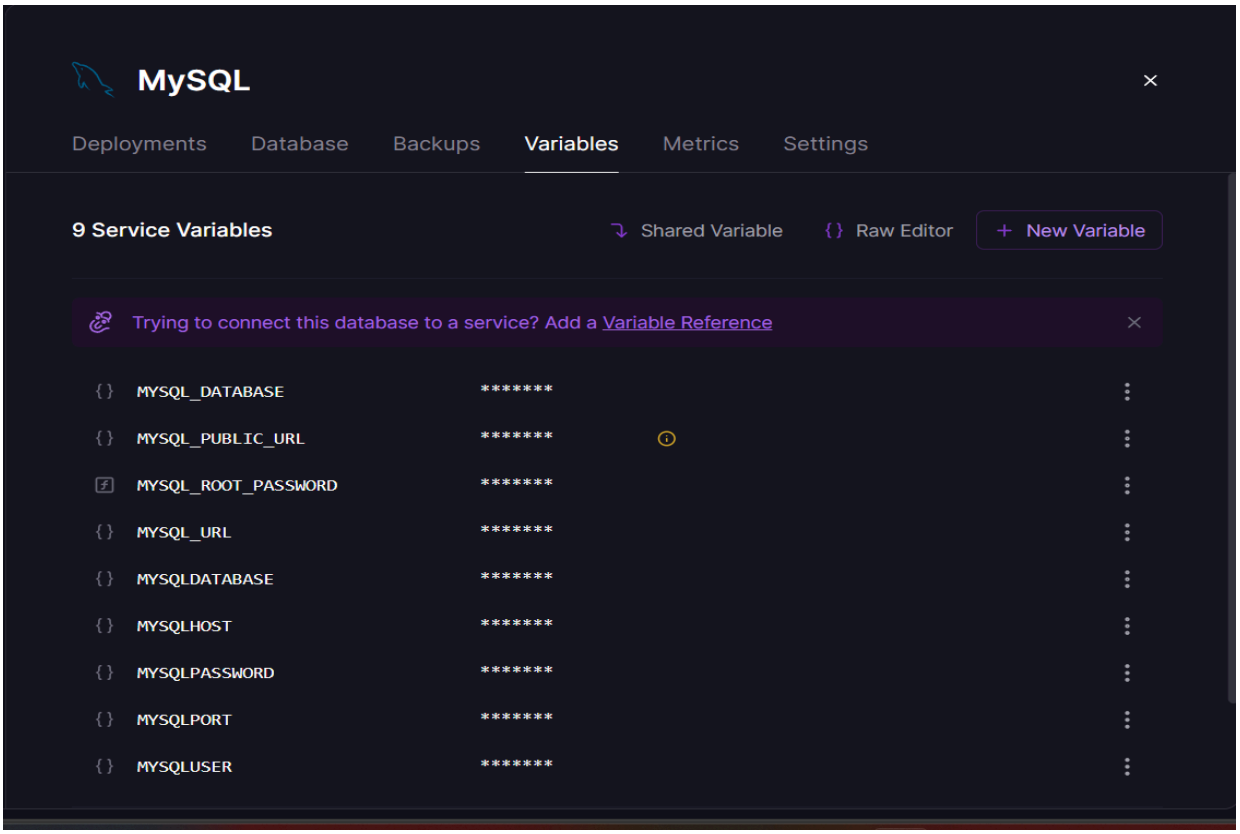


3. Despliegue y Configuración

- **Backend:** Railway App
- **Frontend:** Vercel
- **Configuración:** Uso de variables .env para VITE_API_URL. Ejecución local mediante npm install y npm run dev. El archivo application.properties se adaptó para usar variables de entorno (${MYSQLHOST}, ${MYSQLPORT}, etc.). Dentro del proyecto de Railway, se añadió un servicio MySQL y generó automáticamente las variables de entorno MYSQLHOST, MYSQLPORT, MYSQLUSER, MYSQLPASSWORD, MYSQLDATABASE. Se conectó el repositorio de GitHub a Railway (desde el dashboard: New Project → Deploy from GitHub repo). Railway detectó el proyecto Java y lo compiló con Gradle.

```
spring.datasource.url=jdbc:mysql://${MYSQLHOST}:${MYSQLPORT}/${MYSQLDATABASE}?useSSL=false&serverTimezone=America/Mexico_City&allowPublicKeyRetrieval=true
spring.datasource.username=${MYSQLUSER}
spring.datasource.password=${MYSQLPASSWORD}
```




MySQL
Deployments
Database
Backups
Variables
Metrics
Settings
9 Service Variables
Shared Variable
Raw Editor
New Variable
Trying to connect this database to a service? Add a Variable Reference
MYSQL_DATABASE
MYSQL_PUBLIC_URL
MYSQL_ROOT_PASSWORD
MYSQL_URL
MYSQLDATABASE
MYSQLHOST
MYSQLPASSWORD
MYSQLPORT
MYSQLUSER
****

Parte C. Análisis aplicado (Criterio propio)

C.1 Decisiones reales tomadas

Decisión 1: Separar el catálogo de sucursales por áreas de trabajo (Sistemas, Mantenimiento, Redes).

Justificación: Esto permite que el técnico reciba tickets específicos de su área, evitando saturación de información irrelevante.

Decisión 2: Centralización de la navegación mediante un mapeo dinámico de roles (RBAC).

Justificación: En lugar de hardcodear (escribir fijamente) los menús en el componente de la barra lateral, se optó por crear una estructura de datos centralizada (MENU_ITEMS). Esto permite que la interfaz sea **reactiva al perfil del usuario**: si el backend cambia el rol de un usuario, el frontend adapta automáticamente toda la navegación, iconos y rutas permitidas sin necesidad de modificar los componentes visuales. Esto reduce la probabilidad de errores de seguridad visual (UI) y facilita la escalabilidad del sistema si se agregan nuevos roles en el futuro.

Decisión 3: Autenticación stateless con JWT en lugar de sesiones HTTP.

Justificación: La aplicación está desplegada en Railway y el frontend es independiente (React). Usar sesiones con cookies habría añadido estado en el servidor, dificultando la escalabilidad horizontal. Además, el frontend necesitaba un mecanismo sencillo para enviar credenciales en cada petición. Se optó por JWT (JSON Web Tokens) porque es stateless, el token se guarda en el cliente y se valida en el backend sin necesidad de almacenar sesiones.

```
    1 usage  PerezAquino251243
    private Boolean isTokenExpired(String token) { return extractExpiration(token).before(new Date()); }

    // Generar token para un usuario (email)
    1 usage  PerezAquino251243
    public String generateToken(String username) {
        Map<String, Object> claims = new HashMap<>();
        return createToken(claims, username);
    }

    1 usage  PerezAquino251243
    private String createToken(Map<String, Object> claims, String subject) {
        return Jwts.builder()
                .setClaims(claims)
                .setSubject(subject)
                .setIssuedAt(new Date(System.currentTimeMillis()))
                .setExpiration(new Date(System.currentTimeMillis() + EXPIRATION_TIME))
                .signWith(getSigningKey(), SignatureAlgorithm.HS256)
                .compact();
    }

    // Validar token contra UserDetails (email y expiración)
    1 usage  PerezAquino251243
    public Boolean validateToken(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return (username.equals(userDetails.getUsername()) && !isTokenExpired(token));
    }
}
```

C.2 Contraste o crítica

Propuesta de la IA: La IA sugirió inicialmente una arquitectura de componentes plana y un manejo de estados de tickets basado exclusivamente en el cliente (Frontend), donde toda la lógica de filtrado por roles se ejecutaba después de descargar todos los datos de la base de datos.

Crítica y Desacuerdo Razonado: Se rechazó esta propuesta por dos razones:

1. **Seguridad:** Permitir que un usuario con rol de Encargado descargue en su navegador datos de tickets que no le pertenecen (aunque el Front los oculte) es una vulnerabilidad de seguridad grave. Se decidió que el filtrado debe ocurrir en el Backend mediante el endpoint /scoped.

2. **Escalabilidad:** Una estructura de carpetas plana dificulta el mantenimiento. Se optó por una arquitectura modular separando los servicios de API (servicios), los componentes reutilizables de UI (Badge, Button, Sidebar) y las vistas de página (Estadisticas, Dashboard). Esto permite que el proyecto crezca sin volverse caótico.

C.3 Síntesis y priorización

Tras la implementación de la primera fase funcional del sistema de tickets, se ha definido una hoja de ruta centrada en la optimización del rendimiento y la expansión de funcionalidades administrativas. El objetivo es pasar de una herramienta operativa a una plataforma de análisis de datos técnica.

A corto plazo: Implementación de caché de datos con React Query La primera mejora concreta consiste en integrar una capa de manejo de estado asíncrono para las estadísticas y la lista de tickets. Actualmente, cada cambio de pestaña dispara una petición al servidor; con esta mejora, los datos se mantendrán en caché local, reduciendo el tiempo de carga a casi cero para el usuario y disminuyendo la carga de procesamiento en la base de datos de Spring Boot.

A corto plazo: Dashboard de métricas comparativas por sucursal La segunda mejora es la creación de una vista analítica que permita al Administrador comparar el rendimiento entre diferentes sucursales. Esto implica desarrollar gráficas de barras dinámicas que muestren qué sucursales reportan más incidencias y cuál es su tiempo promedio de respuesta, permitiendo tomar decisiones preventivas sobre el mantenimiento de hardware en sitios específicos.

A corto plazo: Manejo global de excepciones con @ControllerAdvice, Crear una clase GlobalExceptionHandler que capture todas las excepciones comunes (MethodArgumentNotValidException, RuntimeException, AccessDeniedException, etc.) y devuelva una estructura JSON uniforme con código de error y mensaje legible, sin filtrar información sensible. De esta manera se reduce código duplicado, mejora la consistencia de las respuestas de error y evita fugas de información.

Para después: Módulo de generación de reportes ejecutivos en PDF Se ha decidido delegar la generación de reportes formales a una fase posterior debido al balance entre riesgo y esfuerzo.

- **Esfuerzo:** Requiere la integración de librerías pesadas en el servidor (como iText o JasperReports) y el diseño de plantillas que mantengan la identidad visual del proyecto.

- **Riesgo:** Existe un riesgo técnico alto en el manejo de memoria del servidor al generar documentos con múltiples gráficas vectoriales. Dada la prioridad actual de mantener la fluidez del sistema, se considera que el esfuerzo debe invertirse primero en la estabilidad de las consultas antes que en la exportación de documentos, cuya utilidad es administrativa y no operativa inmediata.